# UNIT 1 - Introduction to Data Structures

**Introduction:** Basic Terminology, Elementary Data Organization, Built in Data Types in C. Algorithm, Efficiency of an Algorithm, Time and Space Complexity, Asymptotic notations: Big Oh, Big Theta and Big Omega, Time-Space trade-off. Abstract Data Types (ADT)

**Arrays:** Definition, Single and Multidimensional Arrays, Representation of Arrays: Row Major Order, and Column Major Order, Derivation of Index Formulae for 1-D,2-D,3-D and n-D Array Application of arrays, Sparse Matrices and their representations.

**Linked lists:** Array Implementation and Pointer Implementation of Singly Linked Lists, Doubly Linked List, Circularly Linked List, Operations on a Linked List. Insertion, Deletion, Traversal, Polynomial Representation and Addition Subtraction & Multiplications of Single variable & Two variables Polynomial.

## ◇ 1. What is Data Structure?

- A **Data Structure** is a **way to store and organize data** in computer memory so that it can be used **efficiently**.
- It helps in **performing operations** like searching, sorting, inserting, deleting, and updating data easily.

**Example:**

- Think of a cupboard — arranging clothes properly makes it easier to find them.
- Similarly, data structures organize data for easy access.

**Common Data Structures:**

→ Array, Linked List, Stack, Queue, Tree, Graph, Hash Table

## ◇ 2. Basic Terminology

| Term | Meaning | Example |
|---|---|---|
| **Data** | Raw facts or figures. | 10, 25, “Gautam” |
| **Information** | Processed data with meaning. | “Gautam scored 90 marks” |
| **Data Item** | Smallest unit of data. | Roll number = 101 |
| **Group Item** | Set of related data items. | Student record = {Name, Roll, Marks} |
| **Record** | Collection of related fields. | One student’s data |
| **File** | Collection of related records. | File containing data of all students |
| **Field** | Single attribute of record. | “Name” field in student record |

## ◇ 3. Elementary Data Organization

- Data can be arranged in **two main ways:**

| Type | Description | Example |
|---|---|---|
| **Linear Data Structure** | Data elements arranged in sequence. | Array, Linked List |
| **Non-linear Data Structure** | Data elements connected in hierarchy or network. | Tree, Graph |

**Example:**

- Linear → Students standing in a queue (one after another)
- Non-linear → Family tree (parent–child relationships)

## ◇ 4. Built-in Data Types in C

C language provides **built-in (primitive) data types** used to build complex structures.

| Data Type | Description | Example |
|---|---|---|
| `int` | Integer values | `int a = 10;` |
| `float` | Decimal numbers | `float b = 3.14;` |
| `char` | Single character | `char c = 'G';` |
| `double` | Large decimal numbers | `double d = 10.234;` |
| `void` | Represents no value | Used in functions |

👉 **Note:** All complex data structures (like arrays, stacks, etc.) are made using these basic types.

## ◇ 5. Algorithm

- An **Algorithm** is a **step-by-step process** to solve a problem.
- It takes some **input**, performs **operations**, and gives **output**.

### Example:

Algorithm to find the sum of two numbers

1. Start
2. Read numbers A and B
3. Add them → `SUM = A + B`
4. Print SUM
5. Stop

### Characteristics of a Good Algorithm:

- Has **input(s)** and **output(s)**

- Each step is **clear** and **finite**
- Should **end after some steps**
- Should be **effective and correct**

## ◇ 6. Efficiency of an Algorithm

Efficiency tells how good an algorithm is. It depends on two factors:

| Type | Meaning |
|---|---|
| **Time Efficiency** | How fast the program runs |
| **Space Efficiency** | How much memory it uses |

**Example:**

- Bubble sort takes more time than Quick sort → less time efficient.
- Using extra memory (like temporary array) can increase space usage.

## ◇ 7. Time and Space Complexity

### (a) Time Complexity

→ Time taken by an algorithm to execute completely.

- Depends on **number of operations** and **input size (n)**.

**Example:**

If loop runs n times → Time complexity = **O(n)**

### (b) Space Complexity

→ Total memory required by an algorithm.

Includes:

- Input data size
- Temporary variables
- Recursion stack (if any)

**Example:**

If algorithm uses constant extra space → Space = **O(1)**

## ◇ 8. Asymptotic Notations

Used to measure **performance (growth rate)** of algorithms.

| Notation | Symbol | Meaning | Example |
|---|---|---|---|
| **Big O** | $O(f(n))$ | Upper bound → Worst case | $O(n^2)$ for Bubble Sort |
| **Big Ω** | $\Omega(f(n))$ | Lower bound → Best case | $\Omega(n)$ for Linear Search |
| **Big Θ** | $\Theta(f(n))$ | Tight bound → Average case | $\Theta(n \log n)$ for Merge Sort |

### Example:

For Bubble Sort

- Best case → $\Omega(n)$
- Average case → $\Theta(n^2)$
- Worst case → $O(n^2)$

**Graph Idea:**

If input size increases, execution time also increases — asymptotic notation helps compare algorithms easily.

## ◇ 9. Time–Space Trade-off

- It is a **compromise** between **speed** and **memory**.
- Sometimes we use **more memory** to make the program **faster**, or vice versa.

| Case | Explanation |
|---|---|
| Using lookup tables | Space ↑ → Time ↓ (Faster) |
| Computing every time | Space ↓ → Time ↑ (Slower) |

**Example:**

- Storing pre-computed values of `sin(x)` → saves time but uses memory.
- Calculating `sin(x)` every time → saves space but takes time.

## ◇ 10. Abstract Data Type (ADT)

- **ADT** defines **what operations can be done** on data, not **how** they are done.
- It hides internal details (implementation).

**Example – Stack ADT:**

Operations:

1. `push()` – add element
2. `pop()` – remove element
3. `peek()` – check top element

## ◇ 1. Definition of Array

- An **Array** is a **collection of elements of the same data type** stored in **contiguous (continuous) memory locations**.
- Each element can be accessed using an **index** (position number).

- Index always starts from **0** in C, C++, and Python.

**Example (1-D array in C):**

```
int marks[5] = {90, 85, 78, 88, 92};
```

- Here `marks[0] = 90`, `marks[1] = 85`, etc.
- All 5 values are integers stored one after another in memory.

## ◇ 2. Types of Arrays

### (a) Single Dimensional Array (1-D)

- A **linear list** of elements.
- Accessed using **one index**.

**Example:** `int roll[5] = {1, 2, 3, 4, 5};`

- Roll numbers stored in a straight line (like a row).

**Visualization:**

```
Index →  0   1   2   3   4
Value → [1] [2] [3] [4] [5]
```

### (b) Multi-Dimensional Array

When an array has **more than one dimension** (rows, columns, etc.).

#### *(i) Two-Dimensional Array (2-D)*

- Like a **table or matrix** (rows × columns).

**Example:**

```
int matrix[2][3] = { {1, 2, 3},    {4, 5, 6}  };
```

**Visualization:**

```
Row 0 → [1 2 3]
Row 1 → [4 5 6]
```

- Access using `matrix[row][column]`.

**Example:** `matrix[1][2] = 6`

#### *(ii) Three-Dimensional Array (3-D)*

- Like a **cube of data**.

- Accessed using **3 indexes**.

**Example:**

```
int cube[2][2][2] = {
   {{1,2},{3,4}},
   {{5,6},{7,8}}
};
```

- Access element → `cube[layer][row][col]`

### *(iii) n-Dimensional Array*

- Arrays can have any number of dimensions.
- But in real-world programs, we rarely use beyond 3D.

## ◇ 3. Representation of Arrays in Memory

- Arrays are stored **sequentially** in memory (continuous blocks).
- For **multidimensional arrays**, elements are stored either in:
  - **Row Major Order**
  - **Column Major Order**

### 🧠 (a) Row Major Order (used in C/C++)

- The **complete first row** is stored first, then second row, and so on.

**Example:**

For a 2×3 array

```
A[2][3] = { {1, 2, 3},
            {4, 5, 6} }
```

Stored in memory as:

```
1 2 3 4 5 6
```

### 🧠 (b) Column Major Order (used in FORTRAN, MATLAB)

- The **complete first column** is stored first, then second column, and so on.

**Stored as:**

```
1 4 2 5 3 6
```

## ◇ 4. Derivation of Index Formula

Let

- **Base Address (BA)** = Address of the first element
- **Size** = Number of bytes per element
- **Index starts from LB (Lower Bound)** → usually 0

## ◇ 5. Applications of Arrays

✅ Used where **same-type elements** need to be stored and accessed quickly.

| Application | Example |
|---|---|
| Storing marks of students | `marks[100]` |
| Matrix operations | Addition, Multiplication |
| Searching & Sorting | Linear search, Bubble sort |
| Storing images | 2D pixel arrays |
| Simulation and Data Tables | Sensor readings, daily temperatures |

## ◇ 6. Sparse Matrix

### ➤ What is a Sparse Matrix?

A **sparse matrix** is a **matrix with mostly zero elements**.

Instead of storing all zeros, we store only **non-zero elements** to **save memory**.

**Example:**

```
Matrix:
0  0  5
0  8  0
0  0  0
```

Here only two elements (5, 8) are non-zero.

### ➤ Representations of Sparse Matrix

#### *(a) 3-Tuple (Triplet) Representation*

We store **row, column, and value** of only non-zero elements.

| Row | Column | Value |
|---|---|---|
| 0 | 2 | 5 |
| 1 | 1 | 8 |

We also store matrix size and total non-zero elements.

**Final table:**

| Row | Column | Value |
|---|---|---|
| 3 | 3 | 2 |
| 0 | 2 | 5 |
| 1 | 1 | 8 |

### *(b) Array of Linked Lists*

- Each row can be represented as a **linked list** of non-zero elements.
- Useful for **dynamic** storage.

## ➤ Advantages of Sparse Matrix Representation

1. Saves memory space.
2. Faster processing for matrix operations.
3. Useful in scientific and engineering computations.

## ➤ Disadvantages

1. Complex to implement.
2. Random access becomes difficult.

# ◇ 1. What is a Linked List?

- A **Linked List** is a **collection of nodes** where each node contains:
  - **Data** – actual information
  - **Pointer (link)** – address of the next node
- Unlike arrays, **linked lists do not store elements in continuous memory**.
- Each node is connected using **pointers**.

### 🧱 Structure of a Node in C:

```
struct Node {
    int data;
    struct Node* next;
};
```

## 🧠 Comparison: Array vs Linked List

| Feature | Array | Linked List |
|---|---|---|
| Memory | Continuous | Random |
| Size | Fixed | Can grow/shrink |
| Insertion/Deletion | Costly | Easier |
| Access | Direct (via index) | Sequential (via traversal) |
| Implementation | Simple | Needs pointers |

## ◇ 2. Types of Linked Lists

### (a) Singly Linked List

- Each node has **data** and **next pointer**.
- Last node points to **NULL**.

**Diagram:**

```
[10 | *] → [20 | *] → [30 | NULL]
```

**Structure:**

```
struct Node {
   int data;
   struct Node *next;
};
```

### (b) Doubly Linked List

- Each node has **three parts**:
  - Pointer to previous node
  - Data
  - Pointer to next node

**Diagram:**

```
NULL ← [10 | * | *] ↔ [20 | * | *] ↔ [30 | * | NULL]
```

**Structure:**

```
struct Node {
   struct Node *prev;
   int data;
   struct Node *next;
};
```

**Advantages:**

- Can traverse **both forward and backward**.
- Easier deletion (no need to track previous node separately).

### (c) Circular Linked List

- Last node's next pointer points back to the **first node** (not NULL).
- Can be **singly** or **doubly** circular.

**Diagram (Singly Circular):**

```
[10 | *] → [20 | *] → [30 | *] ↺ (points back to first)
```

**Use:**

- Useful in applications like **round-robin scheduling** or **music playlists**.

## ◇ 3. Implementation of Linked List

### (a) Array Implementation

- We use arrays to simulate links using **index values**.
- Less flexible (fixed size).

**Example:**

```
struct Node {
   int data;
   int next;  // index of next node in array
};
```

### (b) Pointer Implementation

- Each node dynamically created using `malloc()` in C or `new` in C++.
- Most common and flexible method.

**Example:**

```
struct Node* head = NULL;
head = (struct Node*)malloc(sizeof(struct Node));
head->data = 10;
head->next = NULL;
```

## ◇ 4. Operations on Linked List

### (a) Traversal

→ Visiting each node one by one.

**Algorithm:**

1. Start from head.
2. Print data of current node.
3. Move to next node until NULL.

**Example (C Code):**

```
void traverse(struct Node* head) {
    struct Node* temp = head;
    while(temp != NULL) {
```

```
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
```

## (b) Insertion

Adding new node.

**Cases:**

1. At beginning
2. At end
3. After a specific node

**Example (At Beginning):**

```
void insertAtBeg(struct Node** head, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = val;
    newNode->next = *head;
    *head = newNode;
}
```

## (c) Deletion

Removing a node.

**Cases:**

1. Delete from beginning
2. Delete from end
3. Delete a specific value

**Example (Delete from Beginning):**

```
void deleteBeg(struct Node** head) {
    struct Node* temp = *head;
    *head = temp->next;
    free(temp);
}
```

## (d) Searching

- Traverse list until desired data is found.

**Example:**

```
int search(struct Node* head, int key) {
    struct Node* temp = head;
    while(temp != NULL) {
        if(temp->data == key)
            return 1;
        temp = temp->next;
    }
    return 0;
}
```

## ◇ 5. Polynomial Representation using Linked List

A **polynomial** can be written as:

$$P(x) = 7x^3 + 5x^2 + 3x + 9$$

Each term can be stored in a node with:

- **Coefficient**
- **Exponent**
- **Pointer to next term**

### Structure for Polynomial Node:

```
struct PolyNode {
   int coeff;   // coefficient
   int exp;     // exponent
   struct PolyNode* next;
};
```

**Representation in Linked List:**

```
[7 | 3 | *] → [5 | 2 | *] → [3 | 1 | *] → [9 | 0 | NULL]
```

## ◇ 6. Polynomial Operations

### (a) Addition of Two Polynomials

Steps:

1. Traverse both lists simultaneously.
2. If exponents are same → add coefficients.
3. If exponent of one is greater → copy that term.

4. Continue until both lists end.

**Example:**

```
P1 = 7x^3 + 5x^2 + 3x
P2 = 4x^3 + 2x^2 + 6
Result = (7+4)x^3 + (5+2)x^2 + 3x + 6
       = 11x^3 + 7x^2 + 3x + 6
```

## (b) Subtraction of Two Polynomials

Steps same as addition, but subtract coefficients:

P1 - P2 = (7-4)x^3 + (5-2)x^2 + 3x - 6

## (c) Multiplication of Two Polynomials

Steps:

1. For every term in P1, multiply it with all terms of P2.
2. Combine terms with same exponents.

**Example:**

```
P1 = x + 1
P2 = x + 2
Result = x*x + x*2 + 1*x + 1*2 = x² + 3x + 2
```

## (d) Polynomial with Two Variables

Example:

P(x, y) = 3x^2y + 2xy^2 + y

Each node stores:

- Coeff
- Power of x
- Power of y

**Structure:**

```
struct PolyNode2 {
   int coeff, powx, powy;
   struct PolyNode2* next;
};
```